The Complete Synergy Atlas

Every Interaction Between Cubical Type Theory
and Sheaf Cohomology in C^4

CCTT Project

April 16, 2026

Abstract

Cubical type theory and sheaf cohomology are each powerful independently. Cubical type theory gives proof-relevant equality, transport along paths, and higher-dimensional composition. Sheaf cohomology gives local-to-global assembly, decomposition of proof obligations over open covers, and obstruction theory.

The Cubical Cohomological Calculus of Constructions (C^4) is not merely the union of these two theories — it exploits their *interactions*. This document is a comprehensive atlas of **34 synergies** between the cubical and cohomological components, organized by structural level (types, proofs, covers, search, metatheory, Python semantics, trust). Each synergy is classified by whether it provides new Expressivity, Tractability, or Automation.

We prove that the verification power of C^4 is strictly greater than the union of its components.

Contents

We classify each synergy along three axes:

Tag	Meaning
EXPRESSIVITY	Can state or prove things that neither cubical TT nor sheaf cohomology alone can.
TRACTABILITY	Makes an existing proof obligation computationally cheaper (fewer VCs, smaller formulas, lower Z3 complexity).
AUTOMATION	Reduces human annotation burden by synthesizing proof components or search strategies automatically.

1 Level 0: The Foundational Identification

1.1 Synergy 0: Descent = HComp

TAGS: Expressivity, Tractability.

Cubical side. Homogeneous composition (hfill) fills an n-dimensional cube from compatible face data. Given partial proofs on the boundary of $\mathbb{I}^n$, the Kan condition guarantees a filler:

$$\frac{\forall k.\ p_k : \mathsf{Path}_A\big(\partial_k^0(\vec{t}),\ \partial_k^1(\vec{t})\big) \qquad \text{edges agree}}{\mathsf{hfill}^{i_1,\ldots,i_n}\ A\,[\vec{p}]\ t_{\text{base}} : A}$$

Cohomological side. Č descent assembles a global section from local sections on an open cover $\mathcal{U} = \{U_i\}$ with overlap compatibility:

$$\frac{s_i \in \mathcal{F}(U_i) \quad s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}}{s \in \mathcal{F}(X) \text{ with } s|_{U_i} = s_i}$$

The synergy. These are the *same* universal construction:

1. Each fiber φ_k of the refinement cover corresponds to a cubical face $[i_k = \varepsilon]$.
2. The local proof p_k on fiber φ_k is the partial boundary datum on that face.
3. The overlap compatibility $p_i|_{\varphi_i \wedge \varphi_j} = p_j|_{\varphi_i \wedge \varphi_j}$ is the cubical edge agreement condition.
4. The Kan filling condition ("hfill exists") is $H^1(\mathcal{U}, \mathcal{F}) = 0$ for the Č nerve.
5. For propositional coefficients (path types over decidable equality — our setting by Hedberg's theorem), $H^1 = 0$ automatically, so the filling always exists.

Theorem 1.1 (Operational unification). *A single verification-condition generator* `_compile_cubical_filling` *handles both* RefinementDescent *and* HComp *proof terms, differing only in whether faces carry Z3-decidable refinement predicates (descent) or pure cubical names (hcomp).*

Why neither alone suffices.

- Pure cubical TT has no notion of "branching code structure." It can fill cubes, but it does not know that the cube's faces correspond to program branches.
- Pure sheaf theory has no notion of "proof dimension." It can decompose by opens, but has no path structure connecting the local proofs.
- The identification lets the compiler *decompose by branches* (cohomological) and *compose proof paths* (cubical) in a single pass.

1.2 Synergy 0b: The Two Dimension Systems

TAGS: Expressivity.

C^4 has *two independent dimension systems*:

	Cubical dimensions	**Cohomological dimensions**
Parameterized by	Interval variables $i, j, k : \mathbb{I}$	Refinement predicates $\varphi(x)$
0-cells	Programs	Input values
1-cells	Equality proofs (paths)	Fiber restrictions
2-cells	Proof coherence (squares)	Cover overlaps
Composition	hfill (Kan)	Descent (gluing)
Transport	Along type paths	Along predicate implications
Obstruction	Kan condition	$H^1 = 0$
Decidability	Structural	Z3

A proof over a function with n branches and m proof steps lives in $\mathbb{I}^m \times \mathcal{S}^n$ — an $(m+n)$-dimensional space. The cubical dimensions parameterize *how* we prove; the cohomological dimensions parameterize *what* (which inputs) we prove for. The full power of C^4 comes from working in this product space.

2 Level 1: Type-Level Synergies

2.1 Synergy 1: Refinement Types as Cubical Faces

TAGS: Expressivity, Tractability.

A refinement type $\{x : T \mid \varphi(x)\}$ defines both:

- A **subtype** in the type-theoretic sense (elements satisfying φ).
- A **cubical face** in the proof cube (restricting the proof to inputs satisfying φ).

This dual role means that *every type-theoretic refinement is simultaneously a proof decomposition point.* Adding a type annotation to a function parameter automatically provides a cover for decomposition.

Example 2.1. Annotating `def f(x: Positive[int])` gives:

1. A type-level guarantee that $x > 0$ within the body.
2. A cubical face $[\varphi = (x > 0)]$ for the proof.
3. A sheaf restriction $\mathcal{F}|_{\{x>0\}}$ for local verification.

All three are the same data, viewed from different angles.

2.2 Synergy 2: Univalence in the Refinement Lattice

TAGS: Expressivity.

Cubical univalence says: if $A \simeq B$ (as types), then $\mathsf{Path}_{\mathsf{Type}}(A, B)$. Applied to the refinement lattice:

$$\frac{\varphi \Leftrightarrow \psi \quad \text{(Z3 proves equivalence)}}{\mathsf{Path}_{\mathsf{Type}}\big(\{x \mid \varphi\},\ \{x \mid \psi\}\big)}$$

This means: *logically equivalent refinement predicates define the same type*, up to a path. Proofs on $\{x \mid \varphi\}$ can be freely used for $\{x \mid \psi\}$ via transport.

Why this needs both theories.

- The equivalence $\varphi \Leftrightarrow \psi$ is a cohomological fact (Z3-decidable predicate logic on the site).
- The path $\mathsf{Path}_{\mathsf{Type}}(A, B)$ is a cubical fact (univalence axiom).
- Together, they give **Z3-decidable type equalities**: the compiler automatically identifies refinement types with logically equivalent predicates.

2.3 Synergy 3: Fibered Universe Structure

TAGS: Expressivity.

The universe Type in C^4 is *fibered* over the refinement site:

$$\mathsf{Type} \to \mathcal{S}$$

Each type A sits over a refinement predicate φ that describes the inputs for which A is defined. The fiber of Type over φ is the category of types restricted to $\{x \mid \varphi\}$.

Cubical implication: paths in Type are *fibered paths* — they must respect the projection to $\mathcal{S}$. A path between types A and B exists only if their refinement predicates are related (by implication or equivalence).

Cohomological implication: type families over the site are *sheaves of types*. The gluing condition means that locally defined types that agree on overlaps can be assembled into a global type — this is the Glue type of cubical TT, now fibered over the refinement cover.

Remark 2.2. This fibered structure is what prevents "type confusion" in dynamically-typed Python: each branch of an `isinstance` dispatch lives in a *different fiber* of the universe, and proofs cannot accidentally leak between fibers without an explicit transport.

2.4 Synergy 4: Sheafification as Type Completion

TAGS: Expressivity, Automation.

Given a presheaf $\mathcal{F}$ on the site (a type family that does not satisfy the gluing condition), sheafification produces the "best" sheaf $\mathcal{F}^+$ that does. In C^4 terms:

Cohomological reading: sheafification completes a partial type assignment (e.g., a function defined on some but not all branches) to a total one.

Cubical reading: sheafification fills in missing faces of a partial cube — exactly the hfill operation applied to the type level.

Practical consequence: if a function handles some `isinstance` cases but not others, the compiler can *sheafify* the type: automatically extend the type to cover missing cases by filling the cube with $\perp$ (runtime error) or by synthesizing default behavior.

3 Level 2: Proof-Level Synergies

3.1 Synergy 5: Transport Along Refinement Paths

TAGS: Tractability, Automation.

Cubical transport moves proofs along type paths: $\mathsf{transp}^i\, A\, \varphi\, t : A(1)$ given $t : A(0)$.

Applied to refinement types: if we have a proof p that a function f satisfies a property under hypothesis φ_1, and a path (implication) $\varphi_1 \Leftarrow \varphi_2$, then $\mathsf{transp}\, p$ gives a proof of the same property under the *stronger* hypothesis φ_2.

$$\frac{p : P \text{ under } \varphi_1 \qquad \varphi_2 \Rightarrow \varphi_1 \text{ (Z3)}}{\mathsf{transp}(p) : P \text{ under } \varphi_2}$$

Why this needs both.

- The direction of transport (from weaker to stronger hypothesis, or vice versa depending on variance) is a cubical operation.
- The path $\varphi_2 \Rightarrow \varphi_1$ is a cohomological fact (morphism in the site category), checked by Z3.
- The synergy: transport *along morphisms in the site* lets us reuse proofs across different refinement conditions without re-proving.

Example 3.1 (Transport cascade)**.** Given:

- p : "`abs(x) >= 0`" under $\{x > 0\}$
- Z3 proves: $(x > 0) \Rightarrow (x \geq 0)$ and $(x \geq 0) \Rightarrow (x \geq -1)$

Two transports give "`abs(x) >= 0`" under $\{x \geq 0\}$ and $\{x \geq -1\}$ — for free.

3.2 Synergy 6: Restricted Hypothesis Pushing

TAGS: Tractability.

When compiling a fiber proof under predicate φ_k, the predicate itself should be available as a *hypothesis* in the Z3 context. This is:

- **Cubically:** evaluating the proof at face $[i_k = 0]$, which restricts all terms to that face.
- **Cohomologically:** restricting the sheaf to the open set $U_k = \{x \mid \varphi_k(x)\}$, under which φ_k is tautologically true.

Practical impact: without hypothesis pushing, a fiber proof of "$x \geq 0$ under $\{x > 0\}$" would require an explicit proof of $x > 0 \Rightarrow x \geq 0$. With hypothesis pushing, the assumption $x > 0$ is in the Z3 context, so the implication is automatic.

This turns descent from mere *bookkeeping* (split by cases, prove each case from scratch) into genuine *simplification* (split by cases, prove each case with the case hypothesis freely available).

3.3 Synergy 7: Overlap Compatibility = Cubical Edge Agreement

TAGS: Expressivity.

The Č overlap condition says: on $U_i \cap U_j$, the local sections must agree. The cubical edge condition says: on the shared edge of two faces, the partial proofs must agree.

In C^4, an "overlap" is $\varphi_i \wedge \varphi_j$. Whether this is non-empty is decided by Z3 (cohomological). If it is non-empty, the agreement condition is a path equality (cubical):

$$p_i|_{\varphi_i \wedge \varphi_j} =_{\mathsf{Path}} p_j|_{\varphi_i \wedge \varphi_j}$$

Neither theory alone provides this.

- Pure cubical TT checks face agreement but cannot determine which faces overlap (it does not have the Z3-decidable predicate structure).
- Pure sheaf theory requires agreement on overlaps but has no notion of "proof path" to state the agreement as an equality.

3.4 Synergy 8: Automatic Overlap Synthesis

TAGS: Automation.

On non-empty overlaps $\varphi_i \wedge \varphi_j$, compatibility proofs can often be *synthesized automatically* by combining cubical and cohomological reasoning:

1. **Disjointness (Z3):** if $\varphi_i \wedge \varphi_j \equiv \bot$, no compatibility proof is needed. This is the most common case for `if/elif/else` chains.
2. **Containment (Z3 + transport):** if $\varphi_i \Rightarrow \varphi_j$, then the j-proof restricted to the overlap equals the i-proof transported to the overlap. Compatibility is automatic.
3. **Refl $\cap$ Refl (kernel):** if both fiber proofs are reflexivity, their restrictions to the overlap are both reflexivity. Trivially compatible.
4. **Z3 discharge:** if the compatibility obligation is a quantifier-free formula, discharge it directly to Z3 without requiring the user to provide an explicit proof.

Synergy mechanism: strategies 1–2 use the cohomological site structure (Z3 on predicates); strategy 3 uses cubical reflexivity; strategy 4 uses both. The interaction means that *most overlaps in typical Python code are handled without manual proofs.*

3.5 Synergy 9: GluePath — Local Cubical Paths via Descent

TAGS: Expressivity.

The $\mathsf{GluePath}$ proof term glues local *paths* (not just equalities) via Č descent. Each local proof varies over the cubical interval *and* is restricted to a refinement fiber:

$$p_k : \mathbb{I} \to \mathsf{Path}(f|_{\varphi_k},\ g|_{\varphi_k})$$

The descent condition glues these into a global path $p : \mathsf{Path}(f, g)$ that works on all inputs. **This is strictly more powerful than either component:**

- Cubical TT alone can build paths, but cannot decompose them by input cases.
- Sheaf cohomology alone can decompose by cases, but cannot parameterize the local data over a proof interval.
- $\mathsf{GluePath}$ parameterizes the Č data over the cubical interval, giving a "moving" descent.

In the spectral sequence: $\mathsf{GluePath}$ computes the E_1 page, gluing local H^0 path-sections into global H^0.

4 Level 3: Cover-Level Synergies

4.1 Synergy 10: Cover Algebra as Cubical Face Algebra

TAGS: Tractability.

The refinement site has a rich algebra of covers:

$$\begin{aligned}
\text{Refinement:}\quad & \mathcal{U}' \leq \mathcal{U} \ \text{ iff each } \varphi'_j \text{ refines some } \varphi_i \\
\text{Product:}\quad & \mathcal{U} \times \mathcal{V} = \{\varphi_i \wedge \psi_j\} \\
\text{Join:}\quad & \mathcal{U} \vee \mathcal{V} = \{\varphi_i\} \cup \{\psi_j\} \\
\text{Restriction:}\quad & \mathcal{U}|_\psi = \{\varphi_i \wedge \psi\}
\end{aligned}$$

Each of these has a cubical interpretation:

- **Refinement** = subdividing faces of a cube into smaller faces ($\sim$ cubical subdivision).
- **Product** = taking the product of two cubes ($\mathbb{I}^m \times \mathbb{I}^n = \mathbb{I}^{m+n}$).
- **Join** = taking the union of face systems ($\sim$ adding dimensions).
- **Restriction** = evaluating a face map ($\partial_k^\varepsilon : \mathbb{I}^n \to \mathbb{I}^{n-1}$).

All four operations are Z3-decidable (cohomological) and correspond to cubical constructions. No other type theory has this dual algebraic structure on proof decompositions.

4.2 Synergy 11: Product Covers for Interprocedural Composition (Künneth)

TAGS: Expressivity, Tractability.

When function f (proved over cover $\mathcal{U} = \{\varphi_i\}$) calls function g (proved over cover $\mathcal{V} = \{\psi_j\}$), the interprocedural proof uses the product cover:

$$\mathcal{U} \times \mathcal{V} = \{\varphi_i \wedge \psi_j\}$$

The Künneth formula for Č cohomology gives:

$$H^0(\mathcal{U} \times \mathcal{V}, \mathcal{F}) \;\cong\; H^0(\mathcal{U}, \mathcal{F}) \otimes H^0(\mathcal{V}, \mathcal{F})$$

Practical meaning: the composite proof decomposes into *pairs* of fiber proofs. If f has m branches and g has n branches, the monolithic approach needs $O(2^{m+n})$ in the worst case; the product cover needs $O(m \cdot n)$ VCs, and if the covers are independent (many branches don't interact), many product fibers are empty (Z3 detects this), reducing further.

Cubical reading: the product cover corresponds to the product cube $\mathbb{I}^m \times \mathbb{I}^n$. The composite proof fills this product cube from the individual cube-fillings of f and g.

4.3 Synergy 12: Cover Morphisms and Kan Extension

TAGS: Tractability, Automation.

A **cover morphism** $\mu : \mathcal{V} \to \mathcal{U}$ is a map where each $\mathcal{V}$-fiber refines some $\mathcal{U}$-fiber: $\forall j.\ \exists i.\ \psi_j \Rightarrow \varphi_i$.

Given a proof over the coarse cover $\mathcal{U}$, the **left Kan extension** along μ produces a proof over the fine cover $\mathcal{V}$:

$$\mathrm{Lan}_\mu : \mathcal{F}(\mathcal{U}) \to \mathcal{F}(\mathcal{V})$$

Cubically: this is transport along the morphism path in the site category. The proof on each $\mathcal{V}$-fiber is obtained by restricting the proof on the corresponding $\mathcal{U}$-fiber.

Cohomologically: Kan extension is the left adjoint of the restriction functor $\mu^* : \mathcal{F}(\mathcal{U}) \to \mathcal{F}(\mathcal{V})$.

Practical consequences:

1. **Proof reuse under refactoring:** if you proved a function correct with a 3-branch cover, and refactoring splits one branch into two sub-branches, the Kan extension gives you a proof over the 4-branch cover for free (after verifying the cover morphism, which is a Z3 check).
2. **Coarsening for efficiency:** if a fine-grained proof is expensive, prove over a coarse cover and let Kan extension specialize to the finer cover.

4.4 Synergy 13: Connection-Based Cover Simplification

TAGS: Tractability.

The cubical interval $\mathbb{I}$ has **connections** $\wedge_i$ (min) and $\vee_i$ (max), satisfying the De Morgan algebra laws:

$$\neg(i \wedge j) = \neg i \vee \neg j, \qquad \neg(i \vee j) = \neg i \wedge \neg j$$

The refinement lattice has its own $\wedge$ (conjunction) and $\vee$ (disjunction). The **synergy**: these two lattice structures interact to simplify covers via standard lattice identities:

1. **Absorption:** $\varphi \wedge (\varphi \vee \psi) = \varphi$ — collapse nested covers.
2. **Distribution:** $\varphi \wedge (\psi \vee \chi) = (\varphi \wedge \psi) \vee (\varphi \wedge \chi)$ — factor product covers.
3. **Idempotence:** $\varphi \wedge \varphi = \varphi$ — deduplicate fibers.
4. **Complementation:** $\varphi \wedge \neg\varphi = \bot$ — detect dead branches (Z3).

These simplifications reduce the number of VCs generated before any Z3 solving occurs. The cubical De Morgan laws guarantee that the simplifications are *homotopy-invariant* (they preserve the homotopy type of the nerve).

4.5 Synergy 14: Nerve Topology as Proof Connectivity

TAGS: Expressivity.

The Č nerve of a cover $\mathcal{U}$ is the simplicial complex whose k-simplices are $(k{+}1)$-fold overlaps. Its topology encodes the "shape" of the proof decomposition.

Cohomological reading: the nerve's H^1 measures obstructions to gluing. For propositional coefficients, $H^1 = 0$ always (Hedberg), but for *non-propositional* coefficient sheaves (e.g., proof-relevant transport data), H^1 can be non-trivial.

Cubical reading: the nerve embeds into a cubical set (via the standard nerve–cubical adjunction). Its homotopy type determines which hfill operations are possible.

Synergy: a disconnected nerve (components that don't overlap) means the proof *factorizes* into independent sub-proofs. Each connected component of the nerve can be filled independently. This is **proof parallelism** — detected cohomologically, executed cubically.

5 Level 4: Proof Search Synergies

5.1 Synergy 15: Spectral Sequence as Proof Search Strategy

TAGS: Automation.

The Č-to-derived spectral sequence:

$$E_1^{p,q} = \check{\mathrm{C}}^p(\mathcal{U}, \mathcal{H}^q(\mathcal{F})) \;\Rightarrow\; H^{p+q}(X, \mathcal{F})$$

gives a systematic proof search strategy:

Page	Cubical	Cohomological
E_0	—	Extract branch structure from code AST; construct refinement cover $\mathcal{U}$
E_1	Compile each fiber proof (path construction)	Local proofs on each open U_i; local H^0
E_2	Check edge agreement	Overlap compatibility; differential d_1
E_∞	Kan filling (assemble global path)	Gluing into global section

The key synergy: the spectral sequence page structure gives a *priority ordering* for proof search. If E_1 fails (a fiber proof fails), there is no point checking E_2 (overlaps). If E_1 succeeds but E_2 fails, the overlap counterexample identifies exactly which interaction between branches is problematic.

This is strictly more structured than either:

- Cubical proof search (try compositions until one works — no systematic decomposition).
- Cohomological resolution (compute derived functors — overkill for decidable coefficient sheaves).

5.2 Synergy 16: CEGAR via Cover Refinement

TAGS: Automation.

Counter**E**xample-**G**uided **A**bstraction **R**efinement, adapted to the cohomological setting:

1. Start with a coarse cover (e.g., $\{x : \text{int}\}$ — single fiber, no decomposition).
2. Try to prove the property on each fiber.
3. If a fiber proof fails, Z3 provides a **counterexample** (a specific input $x = c$ that violates the property under that fiber's hypothesis).
4. **Split the fiber** at the counterexample: $\varphi_k \mapsto (\varphi_k \wedge x \leq c),\ (\varphi_k \wedge x > c)$.
5. Retry with the refined cover.
6. Iterate until all fibers pass or the maximum depth is reached.

Cubical reading: the counterexample identifies a *point on a cubical face* where the proof fails. The refinement adds a new cubical dimension, splitting the face into two sub-faces that separate the failing case.

Cohomological reading: this is refinement of the cover in the site topology. The refined cover is a finer open covering whose Č cohomology may be simpler (smaller local obligations).

Synergy: the counterexample (Z3, cohomological data) drives the cubical decomposition. Neither theory alone provides this loop: cubical TT has no cover structure to refine, and sheaf theory has no solver to produce counterexamples.

5.3 Synergy 17: Obstruction-Guided Proof Repair

TAGS: Automation.

When the spectral sequence does not degenerate at E_2 (i.e., there are non-trivial differentials), the obstructing cocycle $d_1(s) \in \check{\mathrm{C}}^1(\mathcal{U}, \mathcal{F})$ tells you *exactly which overlaps fail.*

Cubically: a non-trivial d_1 means two cubical faces disagree on a shared edge. The disagreement is a specific pair of face values that do not match.

Cohomologically: the cocycle class $[d_1(s)]$ in H^1 is the obstruction. If it is a coboundary ($d_1(s) = \delta^0 t$), then adjusting the local sections by t makes them compatible.

Practical use: when an overlap compatibility check fails, the compiler can:

1. Identify the specific overlap ($U_i \cap U_j$) that fails.
2. Examine whether the failure is a coboundary (can be fixed by adjusting the local proofs).
3. If so, automatically *repair* the local proofs to achieve compatibility — a form of proof synthesis guided by cohomological obstruction theory.

6 Level 5: Compositional Synergies

6.1 Synergy 18: Transport-Descent Functoriality

TAGS: Tractability, Automation.

Transport and descent commute:

$$\mathsf{transp}\big(\mathrm{desc}(\mathcal{U}, \{p_i\})\big) \;=\; \mathrm{desc}\big(\mathsf{transp}(\mathcal{U}), \{\mathsf{transp}(p_i)\}\big)$$

What this means: if you have a descent proof of f being correct, and a path $q : f \equiv g$ (e.g., from a refactoring or library equivalence), then g is correct via descent over the *same cover structure*, with each fiber proof transported.

Why both theories are needed:

- Transport is cubical (moving along a path in the type/program space).
- Descent is cohomological (decomposition over a cover in the site).
- The functoriality says the two structures are *compatible*: transport preserves the decomposition.

Practical consequence: when a library function is updated (new version with different implementation but same specification), the existing proof transfers automatically. The cost is: 1 path proof (old $\equiv$ new) + 0 re-verification of fiber proofs.

6.2 Synergy 19: Descent Naturality (Proof Independence of Cover)

TAGS: Tractability.

For any cover morphism $\mu : \mathcal{V} \to \mathcal{U}$, descent is *natural*:

$$\mu^*\big(\mathrm{desc}_{\mathcal{U}}(s)\big) \;=\; \mathrm{desc}_{\mathcal{V}}\big(\mu^*(s)\big)$$

This means the result of descent does not depend on the choice of cover (up to the Kan condition). In cubical terms: the filled cube is unique (up to homotopy) regardless of how the faces were decomposed.

Practical impact: you can choose the most *convenient* cover for proving (the one that makes the Z3 obligations simplest), knowing that the result is cover-independent. Different cover choices yield equivalent global proofs.

6.3 Synergy 20: Fiber-Preserving Homotopies (Relative Paths)

TAGS: Expressivity.

A **fiber-preserving homotopy** is a path in the proof space that respects the fibration over the refinement site:

$$H : \mathbb{I} \to \prod_i \mathsf{Path}\big(f|_{\varphi_i},\ g|_{\varphi_i}\big)$$

such that the overlap conditions hold for all $t : \mathbb{I}$.

This is a "moving descent" — a descent proof that varies continuously over the cubical interval. It corresponds to the E_1 page of the spectral sequence: a family of Č 0-cochains parameterized by the interval, satisfying the cocycle condition at each parameter value.

Use case: proving that two algorithms are "homotopically the same" — not just equal on outputs, but equal in a way that is *uniform* over all input branches. This rules out proofs that work "by accident" on some branches.

6.4 Synergy 21: Künneth Decomposition of Nested Proofs

TAGS: Tractability.

When a proof involves *nested* descent (e.g., proving a function that branches, and within each branch calls another function that branches), the Künneth formula decomposes the composite proof:

$$H^0(\mathcal{U} \times \mathcal{V}) \cong \bigoplus_{p+q=0} H^{0p}(\mathcal{U}) \otimes H^{0q}(\mathcal{V}) = H^0(\mathcal{U}) \otimes H^0(\mathcal{V})$$

Cubically: the nested proof lives in a product cube $\mathbb{I}^m \times \mathbb{I}^n$. Künneth says this product cube can be filled from the individual fillings of $\mathbb{I}^m$ and $\mathbb{I}^n$.

Tractability gain: instead of $m \cdot n$ fiber proofs (one for each product fiber), we need only $m + n$ fiber proofs (one for each factor fiber), as long as the interactions are "tensorially simple" (no non-trivial cross-terms in H^1).

7 Level 6: Incremental and Caching Synergies

7.1 Synergy 22: Fiber-Granular Incremental Reverification

TAGS: Tractability.

When code changes in one branch of a function, only the *affected fiber* needs re-verification. The cache key is (predicate hash, proof hash), and the invalidation rule is:

1. Changing fiber φ_k's code invalidates φ_k and all overlaps $\varphi_k \cap \varphi_j$.
2. Changing the cover itself (adding/removing branches) invalidates the exhaustiveness check.
3. Unchanged fibers keep their cached verdicts.

Cubical reading: changing one face of a cube invalidates that face and all edges touching it, but not other faces.

Cohomological reading: this is the Mayer–Vietoris decomposition applied to the caching structure. The long exact sequence:

$$0 \to H^0(X) \to \prod_i H^0(U_i) \to \prod_{i<j} H^0(U_i \cap U_j) \to H^1(X) \to \cdots$$

shows that the global section depends on local sections and overlaps, so invalidating one local section only affects the overlaps it participates in.

7.2 Synergy 23: Transport-Stable Caching

TAGS: Tractability.

If a proof p is cached for predicate φ, and we need a proof for predicate ψ with $\psi \Rightarrow \varphi$ (Z3-verified), we can:

1. Look up p in the cache (cache hit on φ).
2. Apply transport along $\psi \Rightarrow \varphi$ (cubical).
3. The transported proof is valid for ψ without re-running Z3.

This extends the cache to cover not just exact predicate matches but all predicates that *imply* a cached predicate — exponentially increasing cache hit rates for nested or refined predicates.

8 Level 7: Metatheoretic Synergies

8.1 Synergy 24: Decidability Stratification

TAGS: Tractability.

C^4 has a natural decidability stratification:

Level	Decidable by	Theory
Definitional equality	Normalization	Cubical (kernel)
Refinement predicates	Z3	Cohomological (site)
Cover exhaustiveness	Z3 (disjunction)	Cohomological
Overlap emptiness	Z3 (conjunction)	Cohomological
Face agreement	Normalization + Z3	Both
Transport validity	Z3 (implication)	Both
Kan filling	Algorithmic (H^1 check)	Both
Global proof assembly	All of the above	Both

Each level is decidable, but the combination is more powerful than either theory provides alone. The cubical kernel handles structural equality; Z3 handles arithmetic and predicate logic; the cohomological structure tells us *which Z3 queries to issue and in what order.*

8.2 Synergy 25: Complexity Reduction via Decomposition

TAGS: Tractability.

Consider a function with n branches, where each branch's correctness requires a Z3 query of complexity c.

Approach	Formula size	Z3 time
Monolithic (no decomposition)	$O(n \cdot c)$	$O(2^{nc})$
Cohomological (descent)	$n \times O(c)$	$O(n \cdot 2^c)$
+ Cubical (transport reuse)	$k \times O(c)$, $k \leq n$	$O(k \cdot 2^c)$
+ Overlap synthesis	$k \times O(c) + O(k^2)$	$O(k \cdot 2^c + k^2)$

The cohomological decomposition reduces from $O(2^{nc})$ to $O(n \cdot 2^c)$ (exponential in c instead of nc). Cubical transport reduces n to k (only prove dissimilar fibers, transport to the rest). Together, the reduction is from exponential-in-nc to polynomial-in-k times exponential-in-c.

Theorem 8.1 (Decomposition–transport theorem)**.** *For a function with n branches, k distinct proof shapes among them, and per-branch Z3 complexity c:*

$$T_{C^4} \;\leq\; k \cdot T_{Z3}(c) + \binom{k}{2} \cdot T_{Z3}(1) + (n-k) \cdot T_{\text{transport}}$$

where $T_{\text{transport}} = O(1)$ (a single Z3 implication check).

8.3 Synergy 26: Hedberg Collapse

TAGS: Tractability.

Hedberg's theorem: types with decidable equality are sets (all paths between equal elements are themselves equal).

Cohomological consequence: for sheaves valued in sets (our setting: all Python types have decidable equality), all higher cohomology vanishes: $H^n = 0$ for $n \geq 1$.

Cubical consequence: all 2-cubes and higher are automatically fillable (uniqueness of identity proofs).

Synergy: this collapse means:

1. We never need to check H^2 (which would require triple-overlap compatibility).
2. $H^1 = 0$ always, so the Kan filling condition is free.
3. The spectral sequence degenerates at E_1 (all higher differentials vanish).
4. Proof search terminates after checking fibers + pairwise overlaps. No higher-dimensional complications.

This is the reason C^4 is *tractable* for Python: the Hedberg collapse eliminates all the "hard" parts of both cubical TT and sheaf cohomology, leaving only the "easy" parts (paths, covers, overlaps) that are Z3-decidable.

8.4 Synergy 27: Propositional Truncation and Cover Independence

TAGS: Expressivity, Tractability.

Propositional truncation $\|A\|$ turns any type into a proposition (mere truth value). In C^4:

Cubically: truncation collapses all paths to reflexivity. A truncated proof is "the fact that a proof exists" without recording which proof.

Cohomologically: truncation makes the coefficient sheaf *constant* (the same value on every open). The Č cohomology of a constant sheaf on a paracompact space is trivial.

Synergy: when we only need to know that a function satisfies a property (not how), we can truncate all fiber proofs to $\|\text{True}\|$. This makes:

1. All overlaps trivially compatible (constant sheaf).
2. All transport trivial (transport of a constant is the constant).
3. The entire descent reduces to exhaustiveness checking ($\bigvee \varphi_i = \text{True}$), which is a single Z3 query.

9 Level 8: Python-Specific Synergies

9.1 Synergy 28: isinstance Dispatch as Fibration

TAGS: Expressivity.

Python's `isinstance` dispatch:

```python
if isinstance(x, int):      # fiber phi_int
    ...
elif isinstance(x, str):    # fiber phi_str
    ...
else:                       # fiber phi_rest
    ...
```

is simultaneously:

- A cubical face decomposition (three faces of a cube).
- A refinement cover $\{\texttt{isinstance}(x, \texttt{int}),\ \texttt{isinstance}(x, \texttt{str}),\ \neg\texttt{isinstance}(x, \texttt{int}) \wedge \neg\texttt{isinstance}(x, \texttt{str})\}$.
- A sheaf restriction: the type of `x` within each branch is a different fiber of the type sheaf.

Synergy: the compiler uses `isinstance` checks to *simultaneously* construct the refinement cover (cohomological) and the cubical face system (cubical). The type narrowing within each branch (x has type `int` inside the first branch) is exactly the fiber hypothesis pushing of Synergy 6.

9.2 Synergy 29: Dynamic Typing as Coarse-to-Fine Fibration

TAGS: Expressivity, Tractability.

In a dynamically typed language, a variable's type is not known statically. In C^4, this means the variable lives in the *total space* of the type fibration, projected to the coarsest possible cover: $\mathcal{U} = \{\text{True}\}$ (a single fiber covering all inputs).

As runtime checks and assertions narrow the type, the cover is refined: $\{\text{True}\} \to \{\texttt{isinstance}(x, T_1), \neg\texttt{isin}$ ...

Cubically: the refinement adds cubical dimensions one at a time. Each type check adds a new face to the proof cube.

Cohomologically: the refinement is a sequence of cover morphisms $\mathcal{U}_0 \leftarrow \mathcal{U}_1 \leftarrow \cdots$, and Kan extension (Synergy 12) propagates proofs forward through the refinement chain.

Synergy: dynamic typing is not a *problem* for C^4 but a *feature*: it provides a natural sequence of increasingly fine covers, each of which can be verified incrementally.

9.3 Synergy 30: Dict-as-Record Sheafification

TAGS: Expressivity.

Python dictionaries serve as ad-hoc records. In C^4:

Cohomologically: a dictionary with keys $\{k_1, \ldots, k_n\}$ defines a cover of the "key space," and the dict is a section of the presheaf over that cover.

Cubically: accessing key k_i is a face evaluation (∂_{k_i}), and adding/removing keys is a dimensional change.

Synergy: dict operations become sheaf operations:

- `d[k]` = restriction to the open $\{k\}$.
- `d.update(d2)` = gluing two sections on the union of their key covers (requires overlap compatibility on shared keys).
- `d.keys() == d2.keys()` = same cover.
- Dict comprehension = a section defined locally on each key.

9.4 Synergy 31: Exception Handling as Partial Covers

TAGS: Expressivity.

A `try/except` block defines a *partial* cover:

- The `try` block is the "normal" fiber.
- Each `except E` is a fiber restricted to the exceptional case.

Cohomologically: the cover $\{U_{\text{normal}}, U_{E_1}, U_{E_2}, \ldots\}$ may not be exhaustive (unhandled exceptions), and sheafification (Synergy 4) identifies the gap.

Cubically: the exceptional paths are *alternative faces* of the proof cube. The proof decomposes into: normal-case face proof + exceptional-case face proofs.

Synergy: the compiler can:

1. Check cover exhaustiveness: are all exceptions handled? (Z3 on the predicate "`raises(E)`" — requires modeling exception semantics.)
2. If the cover is partial, identify the missing fiber (which exceptions are unhandled).
3. Transport proofs from the normal case to the exceptional case when the exception-handling code preserves the invariant.

9.5 Synergy 32: Generator/Iterator Laziness as Sheaf Sections

TAGS: Expressivity.

A Python generator yields values lazily. In C^4:

Cohomologically: the generator defines a presheaf on the natural number site (indexed by iteration step n). Each "section" is the value yielded at step n.

Cubically: the relationship between consecutive yields `yield`$[n]$ and `yield`$[n{+}1]$ is a path (equality or transformation proof).

Synergy: proving a generator correct means:

1. Per-step fiber proof: for each n, the yield value satisfies the specification (local section).
2. Step-to-step overlap: the transition from step n to step $n{+}1$ is compatible (cubical path + sheaf restriction to the overlap $\{n\} \cap \{n{+}1\}$ in the step space).
3. Induction = descent over the natural number cover $\{0\}, \{1\}, \{2\}, \ldots$ with overlaps at consecutive pairs.

10 Level 9: Trust and Provenance Synergies

10.1 Synergy 33: Trust as Sheaf of Provenance

TAGS: Expressivity.

The trust provenance of a proof is itself a *sheaf* over the refinement cover:

- Fiber φ_k may have trust $\{\text{Z3}\}$ (fully machine-checked).
- Fiber φ_j may have trust $\{\text{LIBRARY}\}$ (depends on library axiom).
- The overlap $\varphi_k \cap \varphi_j$ inherits $\{\text{Z3}, \text{LIBRARY}\}$ (union).

Cubically: composing proofs with different trust levels along a path gives the union of trust sources (trust is a monoid under composition).

Synergy: the trust "map" over the refinement cover tells the user *exactly which parts of the input domain* rely on which trust assumptions. A heat map of trust over the cover: green (Z3) on the easy fibers, yellow (library) on the hard fibers, red (assumed) on the unverified fibers.

10.2 Synergy 34: F*-Style Binding as Fibered Matching

TAGS: Expressivity.

F*-style binding requires that annotations match the code structurally. In C^4, this is a *fibered* matching:

Cohomologically: the annotation's branch structure must match the code's branch structure (same refinement cover).

Cubically: the annotation's proof paths must connect the same endpoints as the code's computational paths.

Synergy: the binding checker verifies:

1. Cover match: the annotation's fibers correspond 1-1 to the code's branches (cohomological).
2. Face match: within each fiber, the annotation's pre/post-conditions match the code's input/output behavior (cubical face evaluation).
3. Path match: the annotation's proof strategy is consistent with the code's control flow (cubical–cohomological: path over the cover).

11 Summary Table

#	Synergy	Expr.	Tract.	Auto.
0	Descent = HComp	✓	✓	
0b	Two dimension systems	✓		
1	Refinement types = cubical faces	✓	✓	
2	Univalence in refinement lattice	✓		
3	Fibered universe structure	✓		
4	Sheafification as type completion	✓		✓
5	Transport along refinement paths		✓	✓
6	Restricted hypothesis pushing		✓	
7	Overlap = edge agreement	✓		
8	Automatic overlap synthesis			✓
9	GluePath (local paths via descent)	✓		
10	Cover algebra = face algebra		✓	
11	Product covers (Künneth)	✓	✓	
12	Cover morphisms + Kan extension		✓	✓
13	Connection-based cover simplification		✓	
14	Nerve topology as proof connectivity	✓		
15	Spectral sequence as proof search			✓
16	CEGAR via cover refinement			✓
17	Obstruction-guided proof repair			✓
18	Transport-descent functoriality		✓	✓
19	Descent naturality		✓	
20	Fiber-preserving homotopies	✓		
21	Künneth for nested proofs		✓	
22	Fiber-granular caching		✓	
23	Transport-stable caching		✓	
24	Decidability stratification		✓	
25	Complexity reduction		✓	
26	Hedberg collapse		✓	
27	Propositional truncation	✓	✓	
28	isinstance dispatch as fibration	✓		
29	Dynamic typing as coarse-to-fine	✓	✓	
30	Dict-as-record sheafification	✓		
31	Exception handling as partial covers	✓		
32	Generator/iterator as sheaf sections	✓		
33	Trust as sheaf of provenance	✓		
34	F*-style binding as fibered matching	✓		
	Total	**20**	**18**	**8**

#	Synergy	Expr.	Tract.	Auto.

Table 1: Complete synergy atlas. Expr. = new expressivity; Tract. = new tractability; Auto. = new automation.

12 The Synergy Theorem

We conclude with the central claim of this document:

Theorem 12.1 (Synergy theorem)**.** *The verification power of* C^4 *is strictly greater than the union of its cubical and cohomological components:*

$$\mathrm{C}^4 \supsetneq \mathrm{CubicalTT} \cup \mathrm{SheafCoh}$$

where $\supsetneq$ *means: there exist programs whose correctness can be stated and proved in* C^4 *but not in either cubical type theory or sheaf cohomology alone.*

Proof sketch. Consider a program f with n branches, where:

1. Branch k has a correctness proof p_k that requires the hypothesis φ_k (the branch condition) to discharge via Z3.
2. The branches share common sub-structure, so that branches i and j have related proofs $p_i \equiv \mathsf{transp}_{\varphi_i \Rightarrow \varphi_j}(p_j)$.
3. The global proof assembles via descent + transport.

Cubical TT alone cannot express the decomposition into branches (it has no site structure to define covers). It would need to prove the property for all inputs monolithically.

Sheaf cohomology alone cannot express the transport between branches (it has no path/interval structure). It can decompose by covers but cannot relate the local proofs to each other except via the overlap condition.

C^4 can: decompose by covers (cohomological), relate branch proofs via transport (cubical), and assemble the global proof via descent = hcomp (the foundational synergy).

The complexity reduction (Synergy 25) shows that this is not merely a notational convenience but a genuine computational advantage: $O(k \cdot 2^c + k^2)$ versus $O(2^{nc})$. □